

Itération : while

Répéter des instructions tant qu'une condition est vraie

Nicholas Wolff · CC BY-SA 4.0

Sommaire

1

Itération

Définition, exemples

Trace d’un algorithme itératif

2

La boucle `while`

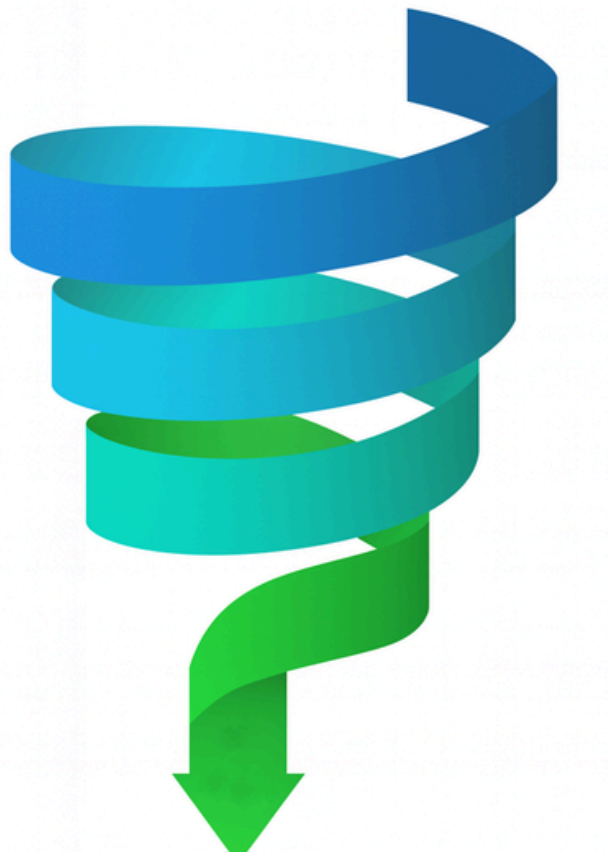
En français : tant que...

Une manière d’itérer en Python

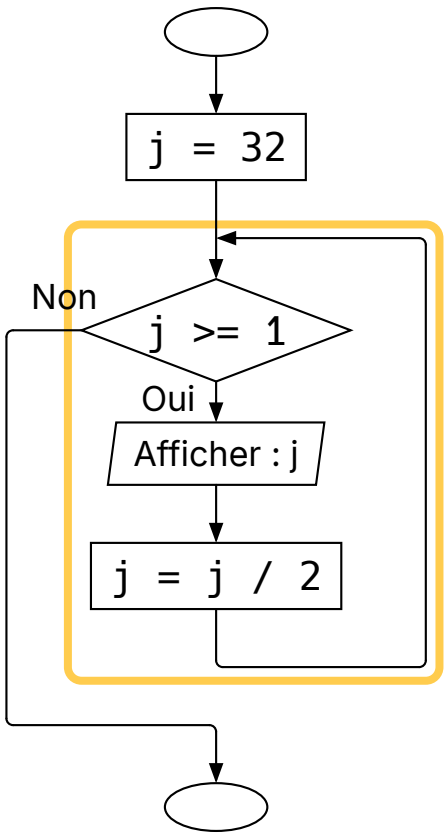
Inversion de condition, technique du drapeau, boucle infinie

Itération

a. Action de répéter, de faire de nouveau ; fait d'être répété.
b. En informatique, procédé de calcul répétitif qui boucle tant qu'une condition particulière est remplie.



Itération : exemple 1



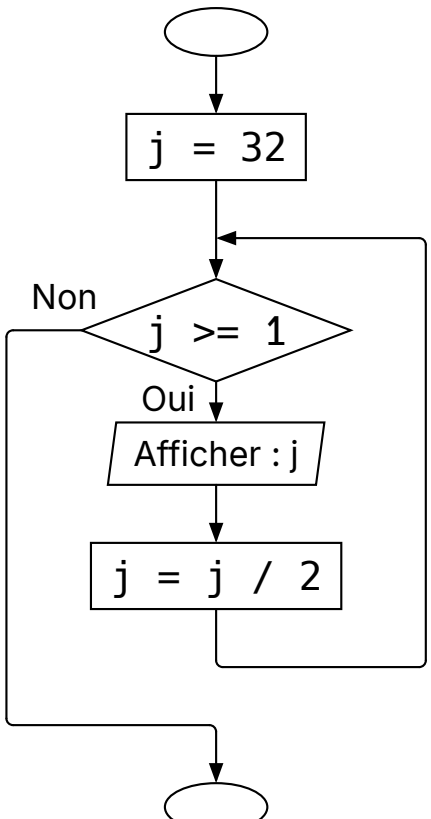
Cet algorithme contient seulement 4 éléments (si on ne compte pas le début et la fin) combinés de manière à ce qu'il y ait une boucle.

Cela veut dire que lors de son exécution, l'algorithme va effectuer bien plus que 4 opérations.

On aimerait écrire une *trace* de l'exécution de l'algorithme, qui indique :

- la manière dont les variables vont changer au fil du temps ;
- ce que l'algorithme affiche.

Rappel : trace d'un algorithme



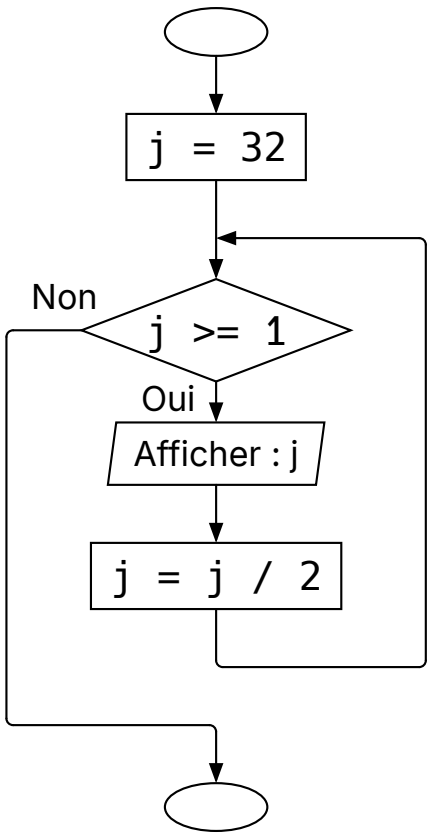
On dessine un tableau avec :

- une colonne par variable ;
- plus une colonne pour ce que l'algorithme affiche.

À chaque fois que la valeur d'une variable change, on ajoute une ligne pour indiquer la nouvelle valeur dans la colonne correspondante.

À chaque fois que l'algorithme affiche quelque chose, on ajoute une ligne pour indiquer ce qui a été affiché.

Rappel : trace d'un algorithme



On dessine un tableau avec :

- une colonne par variable ;
- plus une colonne pour ce que l'algorithme affiche.

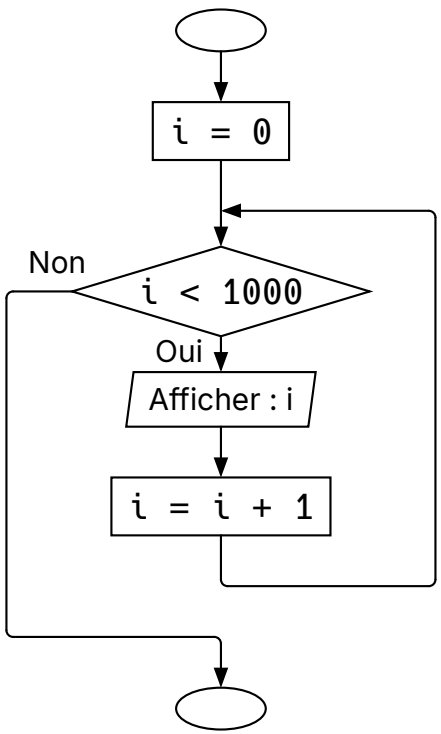
À chaque fois que la valeur d'une variable change, on ajoute une ligne pour indiquer la nouvelle valeur dans la colonne correspondante.

À chaque fois que l'algorithme affiche quelque chose, on ajoute une ligne pour indiquer ce qui a été affiché.

j	affichage
32	
	32
16	
	16
8	
	8
4	
	4
2	
	2
1	
	1
0.5	

temps

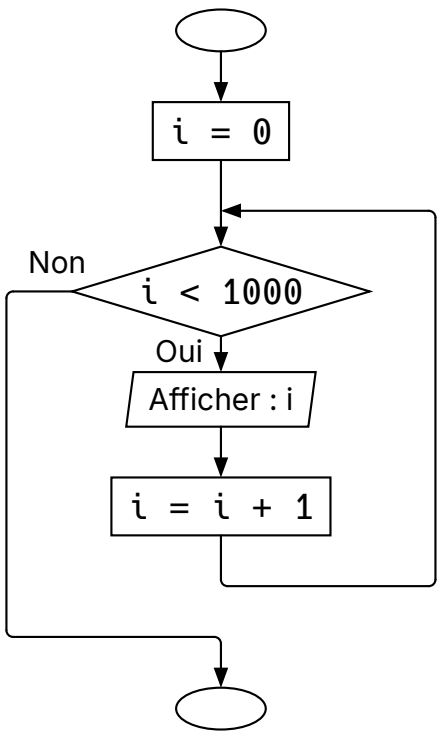
Itération : exemple 2 et exercice



Exercice · 4 min

a) Écrire la trace de cet algorithme.

Itération : exemple 2 et exercice



Exercice · 4 min

a) Écrire la trace de cet algorithme.

i	affichage
0	0
1	1
2	2
…	…
998	998
999	999
1000	

Exercice · 4 min

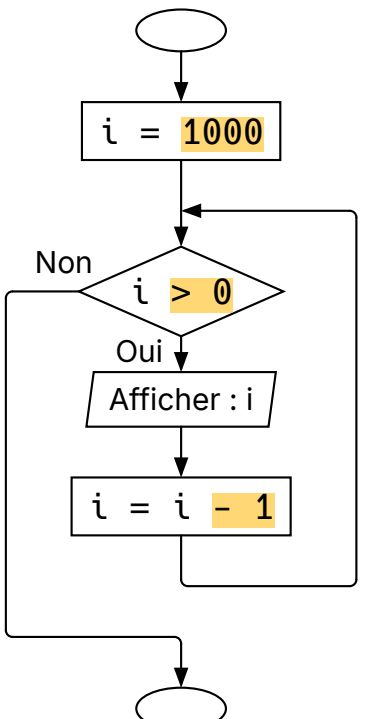
b) Modifier l'algorigramme pour que la trace soit :

i	affichage
1000	1000
999	999
998	998
…	…
2	2
1	1
0	

Itération : exemple 2 et exercice

b) Modifier l'algorigramme pour que la trace soit :

i	affichage
1000	1000
999	999
998	998
…	…
2	2
1	1
0	



Sommaire

1

Itération

Définition, exemples

Trace d’un algorithme itératif

2

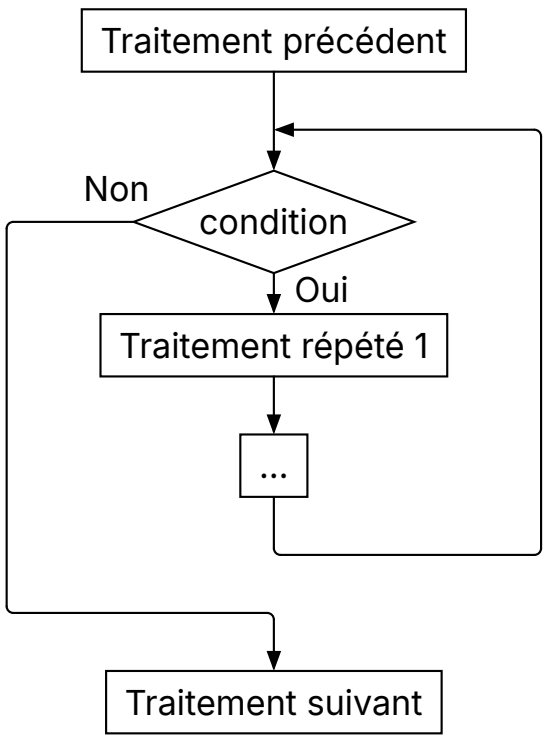
La boucle `while`

En français : tant que...

Une manière d’itérer en Python

Inversion de condition, technique du drapeau, boucle infinie

Itération avec `while`

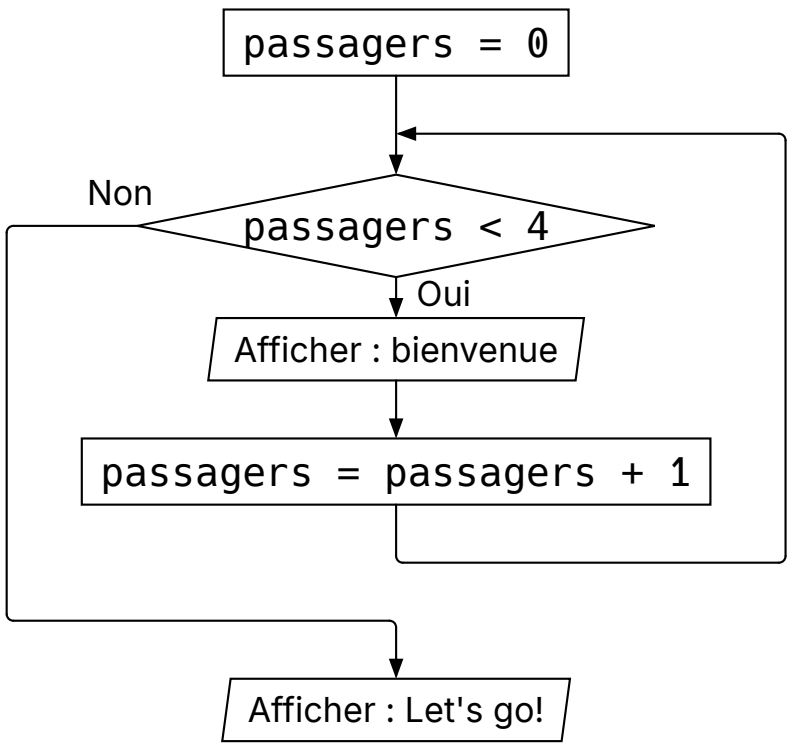


Algorigramme

```
…
traitement_précédent
while condition:
    traitement_répété_1
    …
traitement_suivant
…
```

Programme

Itération avec `while` : exemple en Python



Algorigramme

```
1 passagers = 0
2 while passagers < 4:
3     print("bienvenue")
4     passagers = passagers + 1
5 print("Let's go!")
```

Programme

Itération avec `while` : écrire la trace du programme


```
1 passagers = 0
2 while passagers < 4:
3     print("bienvenue")
4     passagers = passagers + 1
5 print("Let's go!")
```

Exercice · 5 min

Écrire la trace de ce programme.

Contraste `if` – `while`

Voici un programme avec un `if` :

```
1 x = 0
2 if x < 6:
3     print("hello")
4 print("bye")
```

Voici le même programme, dans lequel on a remplacé `if` par `while` :

```
1 x = 0
2 while x < 6:
3     print("hello")
4 print("bye")
```

Contraste `if` – `while`

Voici un programme avec un `if` :

```
1 x = 0
2 if x < 6:
3     print("hello")
4 print("bye")
```

Ce programme affiche :

```
hello
bye
```

Puis il s'arrête.

Voici le même programme, dans lequel on a remplacé `if` par `while` :

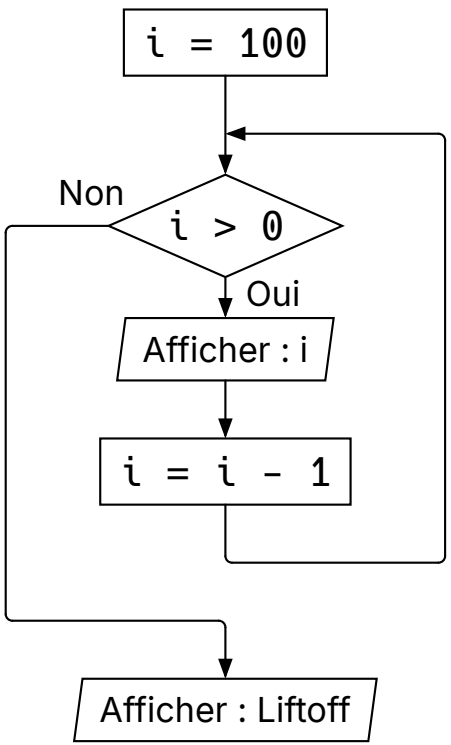
```
1 x = 0
2 while x < 6:
3     print("hello")
4 print("bye")
```

Ce programme affiche :

```
hello
hello
…
```

Et ne s'arrête jamais.

Exercice 1 : algorigramme vers Python



Algorigramme

Exercice · 4 min

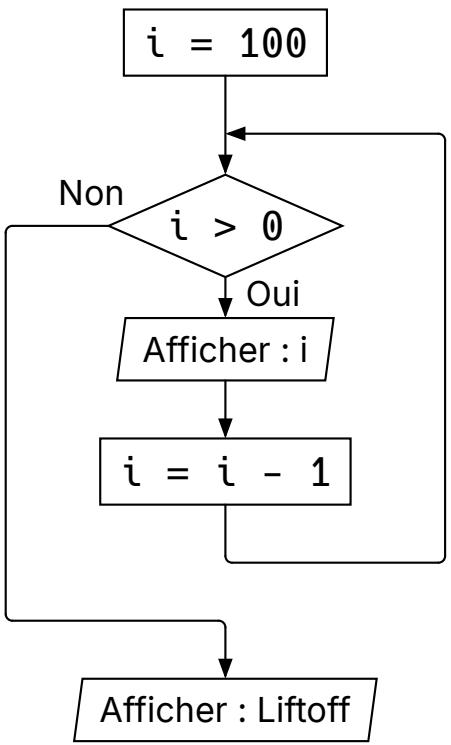
Écrire le programme Python correspondant à l'algorigramme de gauche.

Rappel de la syntaxe :

```
…
traitement_précédent
while condition:
    traitement_répété_1
    …
traitement_suivant
…
```

Programme

Exercice 1 : algorigramme vers Python



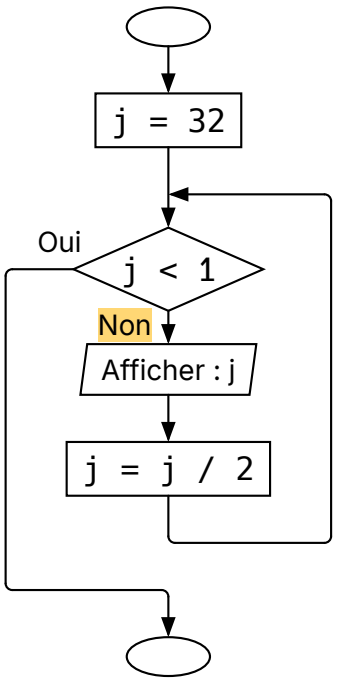
Algorigramme

Écrire le programme Python correspondant à l'algorigramme de gauche.

```python
i = 100
while i > 0:
    print(i)
    i = i - 1
print("Liftoff")
```

Programme

Exercice 2 : inversion de condition

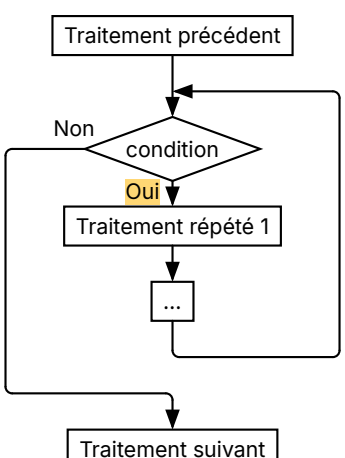



```
32
16.0
8.0
4.0
2.0
1.0
```

Exercice · 5 min

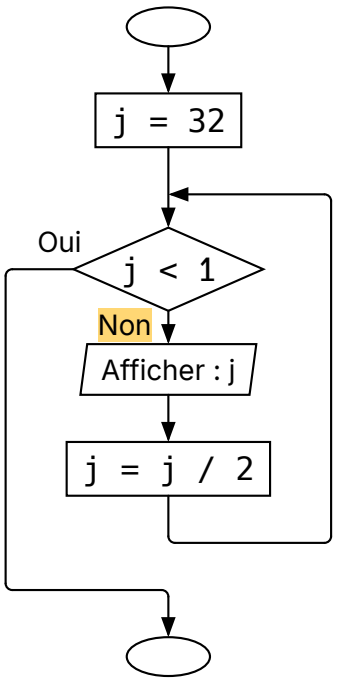
Écrire le programme Python correspondant à l'algorigramme de gauche. On s'attend à l'affichage de la console, au milieu.

Attention : bien regarder où sont le oui et le non sur l'algorigramme. À gauche, la condition nous dit quand il faut **s'arrêter**, alors que dans un `while` il faut écrire une condition qui dit quand on **continue** (on boucle tant que la condition est vraie). Il va donc falloir *inverser la condition* pour écrire le programme.

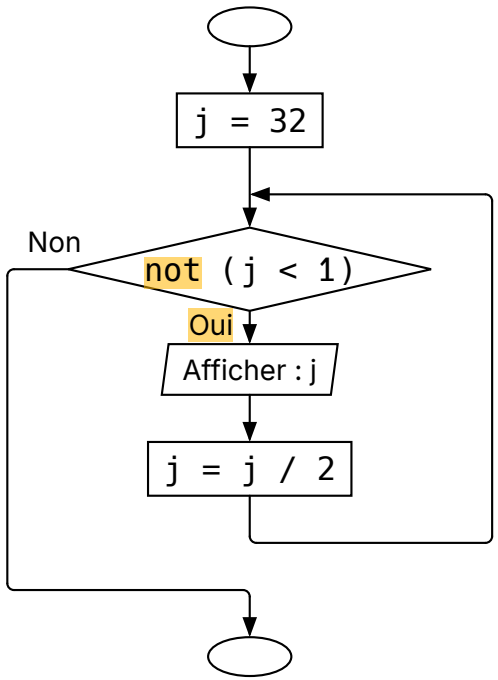



```
…
traitement_précédent
while condition:
    traitement_répété_1
    …
traitement_suivant
…
```

Exercice 2 : inversion de condition



Algorigramme de la consigne



Algorigramme modifié, prêt pour le codage avec `while`

Écrire le programme Python correspondant à l'algorigramme de gauche.

Inversion de la condition

On ne peut pas programmer l'algorithme de la consigne tel quel : dans la condition d'un `while`, il faut répondre « Oui » (`True`) quand on veut *continuer* la boucle.

On garde la condition de la consigne et on l'inverse avec `not` : `not (j < 1)`. On peut aussi réécrire la comparaison : le contraire de `j < 1` est `j >= 1`.

Programme correspondant

Avec `not`, puis avec la comparaison réécrite : les deux programmes font la même chose.

```
1 j = 32
2 while not (j < 1):
3     print(j)
4     j = j / 2
```

```
1 j = 32
2 while j >= 1:
3     print(j)
4     j = j / 2
```

Exercice 3 : multiples de 3

Exercice · 5 min

Écrire un programme qui affiche tous les multiples de 3, à partir de 3 et jusqu'à 60 inclus :

```
3
6
9
…
57
60
```

On ne veut pas de bricolage : ni remplacer un nombre par un autre, ni ajouter une rustine. Par exemple :

- si le programme s'arrête trop tôt, ne pas remplacer 60 par 61, 62 ou 63 dans la condition ;
- ne pas ajouter de `print` pour rattraper ce qui manque.

Exercice 3 : multiples de 3

Écrire un programme qui affiche tous les multiples de 3, à partir de 3 et jusqu'à 60 inclus

On le fait soi-même :

3
6
9
...
57
60

On découvre :

Une solution en Python :

Exercice 3 : multiples de 3

Écrire un programme qui affiche tous les multiples de 3, à partir de 3 et jusqu'à 60 inclus

On le fait soi-même :

3
6
9
...
57
60

On découvre :

- Qu'on a besoin de retenir une seule chose (là où on en est dans l'énumération)
- Qu'on a une itération (on répète tout le temps les mêmes opérations)
- Qu'on commence à 3
- Qu'à chaque fois on ajoute 3
- Qu'on continue tant qu'on n'a **pas dépassé** 60

Une solution en Python :

Exercice 3 : multiples de 3

Écrire un programme qui affiche tous les multiples de 3, à partir de 3 et jusqu'à 60 inclus

On le fait soi-même :

3
6
9
...
57
60

On découvre :

- Qu'on a besoin de retenir une seule chose (là où on en est dans l'énumération)
- Qu'on a une itération (on répète tout le temps les mêmes opérations)
- Qu'on commence à 3
- Qu'à chaque fois on ajoute 3
- Qu'on continue tant qu'on n'a **pas dépassé** 60

Une solution en Python :

```python
m = 3
while m <= 60:
    print(m)
    m = m + 3
```

Exercice 3 : pas de bricolage

Un programme qui s'arrête trop tôt :

```
m = 3
while m < 60:
    print(m)
    m = m + 3
```

Il affiche :

```
3
6
9
…
54
57
```

Bricolage : on remplace 60 par 61.

```
m = 3
while m < 61:
    print(m)
    m = m + 3
```

Bricolage : on ajoute un `print` pour rattraper ce qui manque.

```
m = 3
while m < 60:
    print(m)
    m = m + 3
print(60)
```

Correction : la condition dit « tant qu'on n'a pas dépassé 60 ».

```
m = 3
while m <= 60:
    print(m)
    m = m + 3
```

Exercice 4 : compter de 1 à une valeur donnée

Exercice · 5 min

Écrire un programme qui demande un nombre à la personne qui l'utilise, puis affiche tous les nombres depuis 1 jusqu'au nombre donné, inclus.

Exemple 1 :

- la personne donne le nombre 10 quand on le lui demande ;
- le programme affiche successivement 1, 2, 3, 4, 5, 6, 7, 8, 9, 10.

Exemple 2 :

- la personne donne le nombre 0 quand on le lui demande ;
- le programme n'affiche rien.

Indices

- Rappel : pour demander une chaîne de caractères `v`, on écrit :
  ```
  v = input("un texte pour expliquer ce qu'on demande")
  ```
- Rappel : si on veut demander un nombre entier `u`, on écrit :
  ```
  u = int(input("un texte pour expliquer ce qu'on demande"))
  ```
- Vérifier le programme avec les deux exemples.

Exercice 4 : compter de 1 à une valeur donnée

Écrire un programme qui demande un nombre à la personne qui l'utilise, puis affiche tous les nombres depuis 1 jusqu'au nombre donné, inclus.

On le fait soi-même : **On découvre :** **Une solution en Python :**

On demande un nombre à une personne imaginaire.

On imagine qu'elle répond, par exemple, 5.

1
2
3
4
5

Exercice 4 : compter de 1 à une valeur donnée

Écrire un programme qui demande un nombre à la personne qui l'utilise, puis affiche tous les nombres depuis 1 jusqu'au nombre donné, inclus.

On le fait soi-même :

On demande un nombre à une personne imaginaire.

On imagine qu'elle répond, par exemple, 5.

1
2
3
4
5

On découvre :

- Qu'on a besoin de retenir **deux choses** :
  - la limite fixée par la personne ;
  - là où on en est dans le comptage.
- Qu'on a une itération (on répète tout le temps les mêmes opérations)
- Qu'on commence à 1
- Qu'à chaque fois on ajoute 1
- Qu'on continue tant qu'on n'a **pas dépassé** la limite fixée par la personne

Une solution en Python :

Exercice 4 : compter de 1 à une valeur donnée

Écrire un programme qui demande un nombre à la personne qui l'utilise, puis affiche tous les nombres depuis 1 jusqu'au nombre donné, inclus.

On le fait soi-même :

On demande un nombre à une personne imaginaire.

On imagine qu'elle répond, par exemple, 5.

1
2
3
4
5

On découvre :

- Qu'on a besoin de retenir **deux choses** :
  - la limite fixée par la personne ;
  - là où on en est dans le comptage.
- Qu'on a une itération (on répète tout le temps les mêmes opérations)
- Qu'on commence à 1
- Qu'à chaque fois on ajoute 1
- Qu'on continue tant qu'on n'a **pas dépassé** la limite fixée par la personne

Une solution en Python :

```
1 li = int(input("limite:"))
2 x = 1
3 while x <= li:
4     print(x)
5     x = x + 1
```

Exercice 5 : Batman

Exercice · 5 min

Écrire un programme qui affiche :

```
Na
Na
Na
Na
Na
Na
Na
Batman!
```

Indices

- Na est répété 7 fois, mais il pourrait être affiché 1'000 fois.
- Il n'est pas question de recopier 7 fois la ligne `print("Na")`.
- On peut écrire ce programme en 5 lignes de Python.

Exercice 5 : Batman

Écrire un programme qui affiche : Na (7 fois), puis `Batman!`

On le fait soi-même :

On se concentre sur la partie répétée (afficher `Batman!` à la fin est trivial).

Na (le 1er)
Na (le 2e)
Na (le 3e)
Na (le 4e)
Na (le 5e)
Na (le 6e)
Na (le 7e)

On découvre :

Deux solutions en Python :

Exercice 5 : Batman

Écrire un programme qui affiche : `Na` (7 fois), puis `Batman!`

On le fait soi-même :

On se concentre sur la partie répétée (afficher `Batman!` à la fin est trivial).

Na (le 1er)
Na (le 2e)
Na (le 3e)
Na (le 4e)
Na (le 5e)
Na (le 6e)
Na (le 7e)

On découvre :

- Qu'on a une itération (on répète tout le temps les mêmes opérations)
- Qu'on a besoin de compter
- Qu'on a besoin de retenir une seule chose (là où on en est dans le comptage)
- Qu'à chaque fois dans l'itération on ajoute 1 (ça, c'est la définition de « compter »)
- Qu'on continue tant qu'on n'a pas affiché 7 fois `Na`

Deux solutions en Python :

Exercice 5 : Batman

Écrire un programme qui affiche : Na (7 fois), puis `Batman!`

On le fait soi-même :

On se concentre sur la partie répétée (afficher `Batman!` à la fin est trivial).

Na (le 1er)
Na (le 2e)
Na (le 3e)
Na (le 4e)
Na (le 5e)
Na (le 6e)
Na (le 7e)

On découvre :

- Qu'on a une itération (on répète tout le temps les mêmes opérations)
- Qu'on a besoin de compter
- Qu'on a besoin de retenir une seule chose (là où on en est dans le comptage)
- Qu'à chaque fois dans l'itération on ajoute 1 (ça, c'est la définition de « compter »)
- Qu'on continue tant qu'on n'a pas affiché 7 fois Na

Deux solutions en Python :

En comptant de 1 à 7, comme les Na numérotés à gauche :

```
1 i = 1
2 while i <= 7:
3     print("Na")
4     i = i + 1
5 print("Batman!")
```

Ou en comptant de 0 à 6 : 7 nombres aussi.

```
1 i = 0
2 while i < 7:
3     print("Na")
4     i = i + 1
5 print("Batman!")
```

Exercice 6 : décompte de 100 à 0

Exercice · 3 min

Écrire un programme qui décompte de 100 à 0 : il affiche tous les nombres de 100 à 0 inclus.

```
100
99
…
1
0
```

Exercice 6 : décompte de 100 à 0

Exercice

Écrire un programme qui décompte de 100 à 0 : il affiche tous les nombres de 100 à 0 inclus.

```
100
99
…
1
0
```

Indices

- On peut écrire ce programme en 4 lignes de Python.
- Il est probable qu'on commence par écrire un programme très légèrement faux. Si c'est le cas, bien regarder ce qui ne va pas, et faire les petits ajustements pour arriver à un programme correct.

Exercice 6 : décompte de 100 à 0

Écrire un programme qui décompte de 100 à 0 : il affiche tous les nombres de 100 à 0 inclus.

```
100
99
…
1
0
```

Indices

- On peut écrire ce programme en 4 lignes de Python.
- Il est probable qu'on commence par écrire un programme très légèrement faux. Si c'est le cas, bien regarder ce qui ne va pas, et faire les petits ajustements pour arriver à un programme correct.

Une solution correcte

```
i = 100
while i >= 0:
    print(i)
    i = i - 1
```

Solution incorrecte 1

```
i = 100
while i > 0:
    print(i)
    i = i - 1
```

Solution incorrecte 2

```
i = 100
while i > 0:
    i = i - 1
    print(i)
```

Exercice 7 : nombres impairs de 1 à 999

Exercice · 3 min

Écrire un programme qui affiche les nombres impairs de 1 à 999.

1. Commencer par le faire soi-même sur un papier.

2. Essayer de découvrir :

- de combien de variables on a besoin ;
- s'il y a une itération ;
- si oui, quelle est la valeur de début ;
- ce qu'on fait dans la boucle ;
- on continue tant que …

3. Écrire le programme.

Exercice 7 : nombres impairs de 1 à 999

Écrire un programme qui affiche les nombres impairs de 1 à 999.

On le fait soi-même :

1
3
5
...
997
999

On découvre :

Une solution en Python :

Exercice 7 : nombres impairs de 1 à 999

Écrire un programme qui affiche les nombres impairs de 1 à 999.

On le fait soi-même :

1
3
5
...
997
999

On découvre :

- Qu'on a besoin de retenir une seule chose
- Qu'on a une itération
- Qu'on commence à 1
- Qu'à chaque fois on ajoute 2
- Qu'on continue tant qu'on n'a **pas dépassé** 999

Une solution en Python :

Exercice 7 : nombres impairs de 1 à 999

Écrire un programme qui affiche les nombres impairs de 1 à 999.

On le fait soi-même :

1
3
5
…
997
999

On découvre :

- Qu'on a besoin de retenir une seule chose
- Qu'on a une itération
- Qu'on commence à 1
- Qu'à chaque fois on ajoute 2
- Qu'on continue tant qu'on n'a **pas dépassé** 999

Une solution en Python :

```python
i = 1
while i <= 999:
    print(i)
    i = i + 2
```

Exercice 8 : écrire un programme qui affiche ça

Exercice · 5 min

Écrire un programme qui affiche ça. Le programme commence par demander un nombre ; en bleu, ce que la personne qui l'utilise répond.

Ci-dessous, trois exemples de ce que le programme affiche, pour trois réponses différentes :

```
Un nombre 20
0 20
1 19
2 18
3 17
4 16
5 15
6 14
7 13
8 12
9 11
```

```
Un nombre 12
0 12
1 11
2 10
3 9
4 8
5 7
```

```
Un nombre 0
```

Exercice 8 : écrire un programme qui affiche ça

Écrire un programme qui affiche ça. Le programme commence par demander un nombre ; en bleu, ce que la personne qui l'utilise répond.

Une solution :

```
y = int(input("Un nombre "))
x = 0
while y > x:
    print(x, y)
    x = x + 1
    y = y - 1
```

Exercice 9 : demander jusqu'à l'épuisement

Exercice · 5 min

Écrire un programme qui demande avec insistance à la personne qui l'utilise comment elle va. Le programme s'arrête seulement quand elle répond `Bien`. Juste avant de s'arrêter, le programme affiche `Parfait, au revoir !`

Exemple de conversation (en bleu, ce que la personne qui l'utilise répond) :

```
Comment ça va ? Mal
Comment ça va ? Couci couça
Comment ça va ? Meh
Comment ça va ? Bien
Parfait, au revoir !
```

Indices

- Une petite partie du programme va être dupliquée (une fois en dehors de la boucle, une fois dedans) : il est difficile de faire autrement en Python.
- Il faut boucler tant que la réponse n'est pas `"Bien"` : chercher dans l'aide-mémoire comment on programme ce genre de comparaison.
- On peut écrire ce programme en 4 lignes de Python.

Exercice 9 : demander jusqu'à l'épuisement

Écrire un programme qui demande avec insistance à la personne qui l'utilise comment elle va. Le programme s'arrête seulement quand elle répond `Bien`. Juste avant de s'arrêter, le programme affiche `Parfait, au revoir !`

```
x = input("Comment ça va ? ")
while x != "Bien":
    x = input("Comment ça va ? ")
print("Parfait, au revoir !")
```

Une solution dans laquelle la demande à la personne est dupliquée dans le code.

Exercice 9 : demander jusqu'à l'épuisement

Écrire un programme qui demande avec insistance à la personne qui l'utilise comment elle va. Le programme s'arrête seulement quand elle répond `Bien`. Juste avant de s'arrêter, le programme affiche `Parfait, au revoir !`

```
x = input("Comment ça va ? ")
while x != "Bien":
    x = input("Comment ça va ? ")
print("Parfait, au revoir !")
```

Une solution dans laquelle la demande à la personne est dupliquée dans le code.

```
continuer = True
while continuer:
    x = input("Comment ça va ? ")
    if x == "Bien":
        continuer = False
print("Parfait, au revoir !")
```

On peut éviter de dupliquer la demande en introduisant une variable supplémentaire qui dit s'il faut continuer ou s'arrêter (elle vaut soit `True`, soit `False`).

Cette technique s'appelle la *technique du drapeau* (comme un drapeau dans une course).

Boucle infinie : `while True`

Que fait ce programme ?

```
while True:
    traitement_répété_1
    traitement_répété_2
    …
traitement_suivant
```

Boucle infinie : `while True`

Que fait ce programme ?

```
while True:
    traitement_répété_1
    traitement_répété_2
    …
traitement_suivant
```

La condition `True` fait que la boucle va se répéter sans arrêt : le programme ne va jamais atteindre l'instruction `traitement_suivant`.

Ce genre de boucle apparaît dans tous les programmes interactifs. Par exemple, pour un jeu :

```
while True:
    attendre qu'on appuie sur une touche
    réagir à l'appui de touche
    redessiner l'écran
```

Sources

Contenu

Larousse – définition d’« itération »
larousse.fr

Images

Stephencdickson – les cartes perforées d’un métier Jacquard, National Museum of Scotland (CC BY-SA 4.0)
commons.wikimedia.org